

Extension Guide

Adding a new domain module (backend)

1. Create package under `com.bankone.<module>/` with `controller`, `service`, `repository`, `entity`, `dto` as needed.
2. Wire Spring components (`@RestController`, `@Service`, `@Repository`).
3. Add `SecurityConfig` matchers (do not rely on “authenticated only” for admin mutations).
4. Add entity → verify Hibernate update / add Flyway later.
5. Seed data via `ApplicationRunner` if required.
6. Document: new `docs/MODULES/<Name>.md`, API, schema, call flow, CHANGELOG.

Adding a new Angular feature

1. Generate/create under `src/app/features/<name>/`.
2. Add route in `app.routes.ts` with `canActivate` + `data.roles`.
3. Add sidebar entry in `core/layout/sidebar/` **only when route is real**.
4. Add `core/services/<name>.ts` + models.
5. Update module docs + CHANGELOG.

Extending accounts safely

Goal	Recommended approach
New account type	Add <code>AccountType</code> enum value + policy seed + UI label; do not invent numbers without generator update
Opening deposit change	Prefer <code>account_policy</code> row over hardcoding
Ledger / money move	Already via <code>TransactionService.record</code> on open/deposit/withdraw/transfer; extend types / narrations carefully
Notify on new action	Add publish site in service after save; reuse <code>BankActionEvent</code> + composer (see Notification)
Branch master	New <code>branch</code> table; replace free-text <code>branchCode</code> gradually

Extending security

- Put roles in matchers early.
- Prefer method security `@PreAuthorize` for fine-grained ops.
- Keep JWT secret out of git for production (deferred hardening).

Database evolution path

Today: `ddl-auto=update` + `schema.sql` for sequences.

Target: Flyway/Liquibase migrations under `src/main/resources/db/migration`, set `ddl-auto=validate`.

Stub / thin modules ready for next slices

Module	Package	Suggested first slice
Branch	(none)	Branch master table + validate <code>branchCode</code>
Loan	<code>AccountType.LOAN</code> only	Loan product / policy seed
Notification (Implemented)	<code>com.bankone.notification</code>	Outbox + DLQ (learning plan); notify both transfer parties

Platform learning (not product stubs)

See [TECH_LEARNING_PLAN.md](#) for rate limiting, caching, circuit breaker, idempotency, outbox, versioning, sharding lab, observability, Flyway, etc. Implement one topic per session; update that doc's status column + CHANGELOG.

Already landed (local labs): Redis rate limit, Redis cache-aside ([MODULES/Caching.md](#)), read-replica sync, shard-lab vertical slice.

Already landed product modules (extend carefully): `transaction`, `report`, `audit`, `beneficiary`, `transfer`, `portal`.

When adding a new **cached** read: mark DTO/ `Page` types `Serializable`, add `@Cacheable` + eviction on writers, update `RedisCacheConfig` TTL if needed. ## Documentation template for new features

Every new feature doc must include sections 1–20 listed in `docs/README.md` / architect rule, plus `# Future Modification Guide` with files, classes, methods, impact, and call hierarchy.